Fundamentos teóricos · IoT con Arduino para proyectos escolares

Oct 30, 2026 · @Oscar Enrique Llerena Castro

Cómo usar este documento

Este documento explica la teoría detrás de los siete laboratorios del curso: lo que necesitas entender para conectar, programar y monitorear un dispositivo IoT. Los pasos de cada laboratorio están en la **Guía de laboratorios**; aquí está el porqué.

- Léelo antes del curso o tenlo abierto durante los laboratorios: cada sección dice en qué laboratorio se usa.
- Los valores marcados **(medido en Wokwi)** los obtuvimos en el simulador durante la preparación del curso, en octubre de 2026.
- Los valores marcados **(fabricante)** vienen de la hoja de datos de la pieza: en un aparato real pueden variar.
- Todo el curso usa el simulador gratuito Wokwi (wokwi.com) con una placa ESP32 y el servicio gratuito ThingSpeak (thingspeak.com). No necesitas comprar nada.

1. Qué es el Internet de las Cosas (IoT)

Un dispositivo IoT mide algo del mundo físico, decide qué hacer y envía sus datos por internet para que una persona los vea desde lejos. Todo dispositivo IoT es la misma cadena de cinco eslabones: **sensor → placa → red → nube → persona**. Se usa en todos los laboratorios.



cadena IoT · 5 eslabones y la acción local

Eslabón	Qué hace	En el curso
Sensor	Convierte una magnitud física (temperatura, luz, distancia) en una señal eléctrica	DHT22, BMP180, LDR, PIR, HC-SR04, MQ2
Placa (microcontrolador)	Lee el sensor, ejecuta el programa, decide y mueve un actuador	ESP32 ("un Arduino con WiFi")
Red	Lleva los datos desde la placa hasta internet	WiFi virtual de Wokwi ("Wokwi-GUEST")
Nube	Guarda los datos y los muestra en gráficas	ThingSpeak
Persona	Mira los datos o recibe la alerta, desde cualquier navegador o celular	Tú, el director, el responsable del laboratorio

La placa también puede **actuar** sin esperar a la nube: prender un ventilador, una luz o una alarma. A eso lo llamamos alerta o acción **local**; lo que llega a la persona por internet es la alerta **remota**.

Los cuatro prototipos del curso

Prototipo	Qué mide	Qué decide o hace	Problema escolar
Estación ambiental	Temperatura, humedad, presión y luz	Solo registra y envía	¿Cómo cambia el ambiente del aula durante el día?
Control de temperatura	Temperatura	Prende un ventilador con un relé	Ventilar el aula o el invernadero cuando hace calor
Iluminación inteligente	Luz y movimiento	Prende la luz solo si está oscuro y hay alguien	Luz del pasillo que no queda prendida en vano
Sistema de alerta	Nivel de agua (o gas, o movimiento)	Suena una alarma y avisa por internet	Tanque del colegio que se vacía sin aviso

2. Electricidad básica para conectar piezas

Para que una pieza funcione necesita alimentación (un polo positivo y uno negativo) y, si es un sensor, un cable de señal hacia la placa. Con tres ideas basta para cablear todo el curso. Se usa desde el laboratorio 1.

- **Voltaje (V)**: la "presión" que empuja la corriente. El ESP32 trabaja a **3,3 V**; el Arduino UNO, a 5 V.
- **Corriente (A, mA)**: cuánta electricidad circula. Demasiada corriente quema un LED o un pin de la placa.
- **Tierra (GND)**: el punto de 0 V. **Todas las piezas comparten la misma tierra que la placa** (tierra común); sin ella, la señal no tiene contra qué medirse y la pieza no responde.

Los pines de alimentación del ESP32

Pin	Qué entrega	Para qué se usa en el curso
3V3	3,3 V regulados	DHT22, BMP180, LDR, PIR, MQ2
VIN	5 V del USB	Módulo relé, sensor ultrasónico HC-SR04
GND (GND.1, GND.2)	0 V	El negativo de todas las piezas

Los pines de señal del ESP32 aceptan como máximo 3,3 V (fabricante). En el simulador nada se quema; en una placa real, conectar 5 V a un pin de señal puede dañarla.

Por qué el LED lleva una resistencia

Un LED deja pasar tanta corriente como le den y se quema. La resistencia en serie la limita. La ley de Ohm da el valor:

Corriente = (voltaje del pin − voltaje del LED) ÷ resistencia = (3,3 V − 2 V) ÷ 220 Ω ≈ 6 mA

Con 220 Ω circulan unos 6 mA: el LED brilla y queda lejos de su límite típico de unos 20 mA (fabricante; el 2 V del LED es un valor típico para rojo y amarillo). El LED tiene polaridad: la pata larga (ánodo, **A** en Wokwi) va hacia el pin; la corta (cátodo, **C**) va a GND.

3. La placa: ESP32 frente a Arduino UNO

El ESP32 es una placa que se programa igual que un Arduino, con el mismo lenguaje, pero trae WiFi incorporado: por eso es la placa del curso. El Arduino UNO es la placa clásica de las escuelas y no tiene WiFi. En Wokwi usamos el modelo **ESP32 DevKit V1**.

	Arduino UNO	ESP32 DevKit V1
WiFi	No	Sí
Voltaje de trabajo	5 V	3,3 V
Lectura analógica	0 a 1023 (10 bits)	0 a 4095 (12 bits)
Lenguaje	Arduino (C++)	Arduino (C++), el mismo

Tres tipos de señal

Cada sensor o actuador "habla" con la placa de una de tres formas, y eso decide a qué pin se conecta.

Señal	Qué transmite	Pines del ESP32	Piezas del curso
Digital	Dos estados: 1 (3,3 V) o 0 (0 V), o pulsos	Casi cualquiera (D2, D4, D5, D15, D18, D25, D26, D27…)	LED, relé, zumbador, PIR, HC-SR04, DHT22
Analógica	Un valor continuo de 0 a 3,3 V, leído como 0 a 4095	**Solo D32, D33, D34 y D35** en el curso	LDR (luz), MQ2 (gas)
I2C	Datos por dos cables: SDA (datos) y SCL (reloj)	**D21 (SDA) y D22 (SCL)**	BMP180 (presión)

Por qué los analógicos van en 32 a 35. El ESP32 tiene dos grupos de entradas analógicas; el segundo grupo lo usa el WiFi y no lee mientras el WiFi está encendido (dato de Espressif, el fabricante; no lo probamos en Wokwi). Como todos los prototipos usan WiFi, los sensores analógicos van siempre en pines del primer grupo: D32 a D35 (y D36, D39, que el modelo de Wokwi llama VP y VN). Es un error típico de la IA proponer otro pin.

I2C permite conectar varios sensores a los mismos dos cables: cada sensor tiene una dirección (el BMP180 usa la 0x77, según la documentación de Wokwi). La librería del sensor se encarga de la comunicación.

La tabla completa de conexiones de cada prototipo está en la Guía de laboratorios.

4. Sensores del curso

Cada sensor convierte una magnitud física en una señal; en Wokwi, al hacer clic sobre el sensor durante la simulación, aparece un control (deslizador o botón) para cambiar lo que "siente". Se usan en los laboratorios 1 y 4 a 7.

Sensor	Mide	Señal y conexión	Rango	En Wokwi
DHT22	Temperatura y humedad del aire	Digital, un cable de datos (SDA → D15), 3V3	−40 a 80 °C, 0 a 100 % (fabricante)	Deslizadores de temperatura y humedad; arranca en 24 °C y 40 % (medido en Wokwi)
BMP180	Presión del aire (y temperatura)	I2C: SDA → D21, SCL → D22, 3V3	300 a 1100 hPa (documentación de Wokwi)	Arranca en 1013 hPa, la presión al nivel del mar (medido en Wokwi)
LDR (fotorresistencia)	Luz	Analógica (AO → D34), 3V3	0 a 4095	**El valor sube con la oscuridad**: 1 lux da 3938, 500 lux da 1001, 10 000 lux da 156 (medido en Wokwi)
PIR	Movimiento de personas (calor que se mueve)	Digital (OUT → D27), 3V3	1 = movimiento, 0 = nada	Botón "Simulate motion"; la salida queda en 1 los segundos que diga su atributo delayTime (medido en Wokwi)

Sensor	Mide	Señal y conexión	Rango	En Wokwi
HC-SR04 (ultrasónico)	Distancia	Digital: TRIG → D5 (dispara), ECHO → D18 (escucha), VIN	2 a 400 cm (fabricante)	20 cm se leen como 20,3 cm y 150 cm como 152,1 cm (medido en Wokwi)
MQ2	Gases combustibles y humo	Analógica (AOUT → D35), 3V3	0 a 4095	Solo responde a su deslizador: 843 al mínimo, 3628 con aire limpio, 4041 al máximo (medido en Wokwi)

Cómo miden

- **DHT22**: tiene un sensor de humedad y uno de temperatura, y envía los dos valores ya calculados por un único cable. Necesita la librería "DHT sensor library for ESPx" y entrega una lectura nueva cada 2 segundos como máximo (fabricante).
- **BMP180**: mide la presión atmosférica, que baja con la altura y con el mal tiempo. Necesita la librería "Adafruit BMP085 Library" (el BMP180 es compatible con el BMP085).
- **LDR**: es una resistencia que baja cuando recibe luz. Junto con otra resistencia forma un divisor de voltaje: a más oscuridad, más voltaje en el pin y número más alto. Para que la lectura se entienda la convertimos a porcentaje de luz, donde 100 % es mucha luz: `map(analogRead(34), 4095, 0, 0, 100)`.
- **PIR**: detecta el calor del cuerpo que se mueve frente a él y pone su salida en 1 durante unos segundos. En un PIR real ese tiempo se regula con una perilla.
- **HC-SR04**: emite un pulso de sonido que no oímos y mide cuánto tarda el eco en volver. Como el sonido va y vuelve, la distancia en centímetros es el tiempo en microsegundos dividido entre 58.
- **MQ2**: su resistencia cambia con la concentración de gas. En un aparato real necesita calentarse unos 20 a 30 segundos antes de medir bien (documentación de Wokwi) y debe calibrarse; el valor que da no es una concentración exacta.

5. Actuadores: lo que la placa mueve

Un actuador convierte la decisión del programa en algo visible o audible. La placa lo controla poniendo un pin en 1 (`HIGH`) o en 0 (`LOW`). Se usan en los laboratorios 1, 2 y 5 a 7.

Actuador	Conexión	Cómo se controla	Uso en el curso
LED	Pin → resistencia de 220 Ω → ánodo (A); cátodo (C) → GND	`digitalWrite(pin, HIGH)` lo prende	Aviso, luz del pasillo, "ventilador"
LED RGB	Tres LED en uno (rojo, verde, azul), cada uno con su resistencia	Un pin por color	Semáforo de confort (laboratorio 2)
Módulo relé	VCC → VIN, GND → GND, IN → D26; la carga va entre COM y NO	`HIGH` en IN cierra el contacto COM–NO	Control de temperatura (laboratorio 5)
Zumbador	Pin 2 → D25, pin 1 → GND	`tone(25, 1000)` suena a 1000 Hz; `noTone(25)` lo calla	Alarma (laboratorio 7)

El relé: un interruptor que maneja la placa

El relé es un interruptor que se acciona con la pequeña señal de la placa y puede encender un aparato de mucha más potencia, como un ventilador o una bomba de agua. Tiene dos lados separados:

- **Lado de control** (VCC, GND, IN): se conecta a la placa.
- **Lado de carga** (COM, NO, NC): se conecta al aparato. **COM** es el común; **NO** (normalmente abierto) se une a COM solo cuando el relé se activa; **NC** (normalmente cerrado) está unido a COM cuando el relé está en reposo.

En el modelo del curso, con IN en `HIGH` se prende el LED verde del módulo y el LED "Ventilador" conectado en NO; con IN en `LOW` ambos se apagan (medido en Wokwi). En el simulador la carga es un LED; en la vida real puede ser un aparato de 220 V, y eso exige las precauciones de la sección 10.

6. Programar en Arduino

Un programa de Arduino se llama **sketch** y tiene dos partes obligatorias: `setup()` , que corre una vez al encender, y `loop()` , que se repite sin parar. Antes de correr, el sketch se **compila**: se traduce al lenguaje de la placa. En Wokwi eso pasa al pulsar ▶ y, en el plan

gratis, espera en una cola: el 30 de octubre tardó entre 40 segundos y 3,5 minutos (medido en Wokwi); un proyecto ya compilado arranca en segundos. Se usa en todos los laboratorios.

```
const int PIN_LED = 4;            // constante: un nombre para un valor que
no cambia
const float UMBRAL = 30.0;        // °C

void setup() {                    // corre UNA vez
  Serial.begin(115200);           // abre el monitor serie
  pinMode(PIN_LED, OUTPUT);       // el pin 4 será una salida
}

void loop() {                     // se repite siempre
  float temp = 31.5;              // variable: un valor que cambia (aquí
vendría del sensor)
  if (temp > UMBRAL) {            // decisión
    digitalWrite(PIN_LED, HIGH);
  } else {
    digitalWrite(PIN_LED, LOW);
  }
  Serial.printf("Temp %.1f C\n", temp);  // mensaje al monitor serie
  delay(2000);                    // espera 2 s
}
```

Idea	Para qué sirve	En el ejemplo
Constante (`const`)	Poner nombre a pines y umbrales; se cambian en un solo lugar	`PIN_LED`, `UMBRAL`
Variable	Guardar un valor que cambia	`temp`
`if` / `else`	Decidir	Prender o apagar el LED
Monitor serie	Ver qué está pasando dentro de la placa	`Serial.printf(...)`
Librería	Código ya hecho para un sensor; en Wokwi se agrega en Library Manager (botón azul +)	`#include "DHTesp.h"`

`millis()` en vez de `delay()`

`delay(2000)` detiene la placa 2 segundos: mientras espera no lee sensores ni reacciona. Los prototipos del curso usan `millis()`, que da los milisegundos desde el encendido, para hacer cada cosa a su ritmo sin detenerse: leer cada medio segundo y enviar a la nube

cada 20 segundos.

```
if (millis() - ultimoEnvio >= 20000) {   // pasaron 20 s desde el último
envío
  ultimoEnvio = millis();
  // ... enviar a ThingSpeak
}
```

7. Lógica de decisión: umbral, histéresis y combinaciones

Lo que convierte un medidor en un dispositivo útil es la regla que decide cuándo actuar. El curso usa tres reglas, de menor a mayor complejidad. Se usan en los laboratorios 1, 2 y 5 a 7.

Umbral

Un **umbral** es el valor que separa "normal" de "actúa": si la temperatura pasa de 30 °C, prende el LED. Se elige pensando en el problema (¿a partir de qué temperatura el aula es incómoda?) y se escribe como constante con nombre, para cambiarlo en un solo lugar.

```
bool alerta = temp > UMBRAL;
```

Histéresis: dos umbrales en vez de uno

Con un solo umbral, si la temperatura oscila entre 27,9 y 28,1 °C el ventilador se prende y apaga cada pocos segundos: el relé "tiembla" y se desgasta. La **histéresis** usa dos umbrales: se enciende al pasar el alto y se apaga solo al bajar del bajo. Entre ambos, el ventilador sigue como estaba.

```
const float ENCENDER_SOBRE = 28.0;   // °C
const float APAGAR_BAJO = 26.0;      // °C
if (!ventilador && temp > ENCENDER_SOBRE) ventilador = true;
if (ventilador && temp < APAGAR_BAJO) ventilador = false;
```

Temperatura	Si el ventilador estaba apagado	Si estaba encendido
Sobre 28 °C	Se enciende	Sigue encendido
Entre 26 y 28 °C	Sigue apagado	Sigue encendido
Bajo 26 °C	Sigue apagado	Se apaga

El termostato de una refrigeradora o de una terma funciona así.

Combinar sensores: el "Y" lógico

A veces una sola medida no basta. La iluminación inteligente prende la luz solo si está oscuro **y** hay alguien; en C++ el "y" se escribe `&&` (y el "o", `||`).

```
bool encendida = luz < OSCURO_BAJO_DE && movimiento;
```

Alerta local y alerta remota

El sistema de alerta hace dos cosas con la misma decisión: suena el zumbador y prende el LED en el lugar (alerta **local**), y envía `alerta = 1` a la nube para que alguien lejos se entere (alerta **remota**). La local funciona aunque se caiga internet; la remota avisa aunque nadie esté cerca.

8. Red y nube: cómo viaja el dato

El dato sale de la placa por WiFi, llega a ThingSpeak como una dirección web y queda guardado en un canal que se ve como gráfica. Se usa desde el laboratorio 1.

WiFi en el simulador

Wokwi ofrece una red WiFi virtual llamada **Wokwi-GUEST**, sin contraseña, con salida real a internet: `WiFi.begin("Wokwi-GUEST", "", 6);`. Conectar tardó entre 3 segundos y más de 2 minutos en nuestras pruebas (medido en Wokwi). Si no conecta en un minuto, detén la simulación y vuelve a pulsar ▶.

El envío: una petición HTTP

La placa envía los datos pidiendo una dirección web, igual que un navegador. Los valores van en la misma dirección:

```
http://api.thingspeak.com/update?api_key=TU_CLAVE&field1=24.0&field2=40.0
```

ThingSpeak responde con un código y un número. En el monitor serie lo verás así:

Lo que ves	Qué significa	Qué hacer
`HTTP 200, respuesta 15`	Llegó: es el dato número 15 del canal	Nada: mira la gráfica

Lo que ves	Qué significa	Qué hacer
`HTTP 200, respuesta 0`	Llegó pero no se guardó: enviaste antes de 15 segundos desde el dato anterior; pasa al reiniciar la simulación (medido en Wokwi)	Nada: el siguiente envío, 20 s después, sí se guarda
`HTTP 400`	Clave mal pegada o falta; o un valor salió con un espacio, por ejemplo String(nivel, 0) con un número de una cifra (medido en Wokwi)	Copia otra vez la Write API Key; si la clave está bien, usa String(lround(valor)) o String(valor, 1)
`HTTP -1` u otro negativo	No hay conexión	Revisa que el WiFi diga OK

ThingSpeak: canal, campos y clave

- **Canal**: un tablero de datos de un dispositivo. Cada prototipo tiene el suyo: "Estación CL#-####", "Control CL#-####", etc.
- **Campos** (field1 a field8): las variables del canal. Cada campo tiene su gráfica.
- **Write API Key**: la clave que permite escribir en tu canal; está en la pestaña "API Keys". Quien la tenga puede enviar datos a tu canal.
- **Canal público**: cualquiera con el enlace ve las gráficas, sin cuenta (pestaña "Sharing"). Así se monitorea desde el celular, y así se califica el curso.

Límites del plan gratis (FAQ oficial de ThingSpeak)

Límite	Valor	Consecuencia en el curso
Canales por cuenta	4	Uno por prototipo, exactamente
Intervalo mínimo entre envíos	15 segundos	Los sketches envían cada 20 s
Mensajes por año	3 millones	Un prototipo enviando cada 20 s usa unos 4300 al día
Campos por canal	8	Sobra: el que más usa es la estación, con 4

Otra forma de monitorear, que verás en una demo, es **MQTT**: la placa publica cada dato y quien esté suscrito lo recibe al instante, sin historial.

9. IA para diseñar y programar

La IA (Gemini en el curso) escribe tablas de conexiones, diagramas y sketches en segundos, pero se equivoca de formas que compilan sin error: por eso todo lo que entrega se revisa y se prueba antes de darlo por bueno. Se usa en los laboratorios 2 a 7.

La fórmula RCTFR

Letra	Qué le dices	Ejemplo para un prototipo
Rol	Quién debe ser	"Eres diseñador de prototipos IoT con ESP32 en el simulador Wokwi."
Contexto	Qué tienes	Placa wokwi-esp32-devkit-v1, piezas disponibles con sus pines, pines ya usados
Tarea	Qué debe hacer	"Diseña una luz que se prenda solo si está oscuro y hay movimiento."
Formato	Cómo lo quieres	(1) tabla pieza → pin → tipo de señal; (2) diagram.json; (3) sketch completo comentado en español
Restricciones	Qué no debe hacer	Analógicos solo en D32 a D35; resistencia de 220 Ω con cada LED; GND común; solo piezas y librerías de la lista

Lista de revisión antes de pegar

1. Cada pin de la tabla existe en el ESP32 y no se repite.
2. Los sensores analógicos están en D32, D33, D34 o D35.
3. Cada pieza tiene alimentación (3V3 o VIN) y GND.
4. Cada LED tiene su resistencia.
5. Los pines del sketch son los mismos de la tabla y del diagrama.
6. Las librerías existen en el Library Manager de Wokwi.
7. Los umbrales tienen sentido para el problema y la dirección es correcta (¿mayor o menor?).
8. **Pruébalo**: mueve el deslizador a un valor que debería activar y a uno que no, y mira si pasa lo esperado.

Un error real de la IA

En la preparación del curso le pedimos a Gemini la luz inteligente. Su diagrama abrió bien en Wokwi y su código compiló sin errores. Pero supuso que el sensor de luz da valores bajos en la oscuridad (`valorLuz < 1000` = oscuro), cuando en Wokwi es al revés: 1 lux da 3938 y 10 000 lux dan 156 (medido en Wokwi). Resultado: la luz se habría prendido de día y nunca de noche. Solo el paso 8 de la lista lo descubre.

Si Wokwi muestra un error al compilar, copia el mensaje en rojo y pégaselo a la IA en la misma conversación: "Me sale este error, corrígelo sin cambiar los pines".

10. Seguridad eléctrica y de datos

En el simulador nada se quema ni se filtra, pero al llevar un prototipo al colegio aparecen dos riesgos: la electricidad de la red y los datos en internet. Se trata en el bloque 4.

Seguridad eléctrica en el montaje real

- Alimenta la placa por USB (5 V) desde una computadora o un cargador de celular: es el montaje seguro para el aula.
- Los pines de señal del ESP32 aceptan 3,3 V como máximo (fabricante): no conectes salidas de 5 V a un pin de entrada.
- **220 V es peligroso.** Un relé puede encender un aparato de la red eléctrica, pero ese cableado lo hace solo una persona capacitada, con un módulo relé aislado, en caja cerrada y sin que los estudiantes toquen esa parte. Para el aula, usa cargas de bajo voltaje: un ventilador o una bomba de 5 V.
- Desconecta la alimentación antes de cambiar un cable.

Seguridad de los datos

- **Ningún dato personal en la nube**: ni nombres, ni DNI, ni fotos, ni notas de estudiantes (Ley 29733 de protección de datos personales). Los prototipos del curso solo envían medidas del ambiente.
- **Tus proyectos de Wokwi son públicos** en el plan gratis: cualquiera que abra el enlace ve tu sketch y, en él, tu Write API Key. Usa canales de prueba y, al terminar el curso, genera una clave nueva en la pestaña "API Keys".
- **Tus canales de ThingSpeak serán públicos** para calificarlos y verlos desde el celular: nómbralos con tu código (CL#-####), no con tu nombre.
- Al pedir código a una IA, no pegues claves ni datos de personas.

11. Glosario

Término	Significado
Actuador	Pieza que la placa mueve o enciende: LED, relé, zumbador
API Key (Write API Key)	Clave de ThingSpeak que permite escribir datos en un canal
Canal	Tablero de ThingSpeak donde se guardan los datos de un dispositivo
Campo (field)	Una variable dentro de un canal; cada uno tiene su gráfica
Compilar	Traducir el sketch al lenguaje de la placa antes de correrlo
diagram.json	Archivo de Wokwi que guarda las piezas y los cables del diagrama
ESP32	Placa con WiFi que se programa como un Arduino
GND (tierra)	El punto de 0 V, común a todas las piezas
Histéresis	Usar dos umbrales (encender y apagar) para que el actuador no tiemble
HTTP	El idioma de la web; la placa lo usa para enviar datos a ThingSpeak
I2C	Conexión de datos por dos cables (SDA y SCL), compartida por varios sensores
Librería	Código ya hecho para usar un sensor; se agrega en Library Manager
Monitor serie	Ventana de Wokwi donde la placa escribe mensajes
Relé	Interruptor que la placa acciona para encender un aparato aparte
Sensor	Pieza que convierte una magnitud física en señal eléctrica
Señal analógica	Valor continuo, leído de 0 a 4095 en el ESP32
Señal digital	Solo dos estados: 1 o 0

Término	Significado
Sketch	El programa de Arduino
Umbral	Valor que separa "normal" de "actúa"
Wokwi	Simulador gratuito de placas y piezas en el navegador